

KMI/PRKL - Překladače

Poznámky z výuky (2025 – 2026)
Verze z 04. 06. 2026

Vojtěch Netrh
vojtanetrh@gmail.com

Obsah

1	Režijní informace	4
1.1	Státnicové okruhy	4
1.2	Shrnutí pojmů ke zkoušce	4
2	Historie vývoje	5
3	Překlad programu	5
3.1	Front End	6
3.2	Optimalizace	6
3.3	Back End	6
3.4	Křížový překladač	6
3.5	Tabulka symbolů	6
3.6	Interpret	7
3.7	Chyby při překladu	7
3.7.1	Zotavení při analýze shora-dolů	7
3.7.2	Zotavení při analýze zdola-nahoru	8
4	Regulární gramatiky	8
4.1	Regulární výrazy	9
5	Lexikální analýza / analyzátor	9
5.1	Interní forma posloupnost symbolů	10
5.2	Rozpoznání konce lexikálního symbolu	10
5.3	Klíčová slova v programu	10
5.4	Chyby rozpoznávané při lexikální analýze	11
6	Syntaktická analýza	11
6.1	Syntaktická analýza shora-dolů (LL)	11
6.1.1	Expanze	11
6.1.2	Srovnání	11
6.1.3	Konstrukce zásobníkového automatu pro gramatiku LL(1)	12
6.1.4	Činnost automatu	12
6.2	Konstrukce analyzátoru metodou rekurzivního sestupu	13
6.2.1	Problémy	14
6.3	Syntaktická analýza zdola-nahoru	14
6.3.1	Konstrukce automatu pro SLR(1) gramatiku	15
6.3.2	Konstrukce automatu pro LALR(1) gramatiku	16
6.3.3	Nedeterminismus automatu	17
7	Sémantická analýza	17
7.1	Atributová gramatika	18
7.1.1	Dobře definovaná gramatika	19
7.1.2	L-atributová gramatika	19
7.1.3	Porovnání atributových gramatik při syntaktické analýze	19
7.1.4	S-atributová gramatika	20
7.2	Interní formy programu po sémantické analýze	20
7.2.1	Stromová interní forma (AST ¹)	20

¹Abstract syntax tree

7.2.2	Lineární forma	20
7.2.3	Generování interní formy	21
8	Tabulky symbolů	22
9	Blok programu	23
10	Optimalizace programů	23
10.1	Co vše můžeme optimalizovat	23
11	Generování kódu	24
11.1	Přidělování registrů	24
11.1.1	Přidělování metodou barvení grafu	24

1 Režijní informace

Vyučující: Arnošt Večerka

Zkouška: ústní; otázky buď podle probraných témat nebo na konci semestru

Zápočet: interpret příslušného programovacího jazyka (konkrétně Pascal)

Materiály: dostupné z [cloudu katedry informatiky](#)

1.1 Státnicové okruhy

1. Základní struktura překladače.
2. Fáze analýzy a syntézy překladu.
3. Lexikální analýza, její úloha a konstrukce lexikálního analyzátoru.
4. Syntaktická analýza shora-dolů, gramatiky LL(1).
5. Konstrukce syntaktického analyzátoru metodou rekurzivního sestupu.
6. Syntaktická analýza zdola-nahoru.
7. Konstrukce syntaktického analyzátoru pro gramatiky SLR(1), LALR(1).
8. Sémantická analýza.
9. Atributové gramatiky a jejich specifické typy pro analýzu shora-dolů a analýzu zdola-nahoru.
10. Interní formy programu.
11. Překlad základních příkazů programovacích jazyků do interní formy.
12. Tabulky symbolů.
13. Metody lokální a globální optimalizace programu.
14. Generování kódu, úloha přidělování registrů.

1.2 Shrnutí pojmů ke zkoušce

Zkouška je spíše praktickou formou, takže se řeší příklady dělané na cvičeních, proto je podle mě dobré si k nim sepsat pár poznámek a zjednodušení jak začít a co dělat.

First() ... množina kterou začínám počítat ze zadaného řetězce a zajímá mě jaké všechny terminály mohou dostat na začátku řetězce

Follow() ... začínáme od počátečního řetězce a zajímá mě co všechno (terminály) může následovat za daným neterminálem

LL(1) je gramatika pokud splňuje disjunkci FirstFollow množin

Konstrukce automatu ... očíslovat pravidla, pro každé pravidlo vypočítat FirstFollow množinu a sestavit tabulku (řádky jsou neterminály a sloupce terminály), čísla vypíšeme do políček podle množin FirstFollow (některá mohou být prázdná)

Analyzátor pomocí rekurzivního sestupu ... musím zjistit zdali je LL(1), takže číslování a množiny FirstFollow, pak jedu parse pro každý neterminál a v něm je switch pro jednotlivé terminály z FirstFollow, pro terminály se používá srovnani() a pro neterminály parseN(), srovnani je definováno jako if (u=v) lex() a na začátku každého case máme u=lex()

Lokální optimalizace ... to kde se sestavují interní čtveřice a konstantní výrazy;

1. nejdříve se udělá tabulka pro všechny proměnné s hodnotami **const** na **FALSE** a **num**, kde jsou postupně čísla

2. pak se postupně projdou příkazy a sestaví se čtveřice a další proměnné se číslují jako pokračování **num** (kromě vyjímek)
3. nakonec se čtveřice zjednoduší, aby se neodstranily nepotřebné proměnné

2 Historie vývoje

- Operační systémy
- Pouze instrukce (operační kódy) a čísla (uložené v paměti)
- Soubory s instrukcemi nebyly dříve tak bohaté
- Vylepšení ve formě Assembleru
 - návěští
 - označení instrukcí slovy
 - nutnost překladu ⇒ musíme vymyslet a používat překladač
- Další fází byly programovací jazyky
 - přenositelnost programu mezi systémy (procesory)
 - musí být příslušný překladač
 - překlad je mnohem složitější ⇒ přeložený program je méně efektivní oproti přímému zápisu
- **Fortran** jako první programovací jazyk
- K němu vytvořený překladač byl kvalitní (měl i optimalizace)
- Sestavovací program
- Jazyky **C**, **C++**
- Celý postup překladu
 - preprocesor
 - linker
 - objektový kód
 - strojový kód
 - :
- Specifické interní reprezentace
 - bytecode pro **Java** nebo CIL pro **C#**
 - jsou nezávislé na cílové platformě
 - JIT (just in time) překladač
- 2 základní formy překladače
 1. interní forma – posloupnost symbolů
 2. interní forma – nejběžnější jsou AST (abstraktní syntaktický strom) a lineární forma (čtveřice)

3 Překlad programu

Překlad má obvykle 3 fáze (průchody) – front end, optimalizace, back end. V prvních průchodu se udělá analýza zdrojového programu a vygeneruje se 2. interní forma. V druhém probíhá optimalizace. Ve třetím se už generuje cílový program.

Poznámka 1

Assembler měl překlad dvoufázový.

[Možná vložit obrázek]

První 2 fáze jsou nezávislé na cílové platformě. Pro různé platformy je tedy možné mít první 2 části společné a měnit jen část 3. (Back End).

3.1 Front End

Tato první část je závislá na zdrojovém jazyku. Dělí do 3 částí:

- lexikální,
- syntaktická,
- sémantická analýza.

Lexikální analýza pro atomické části programu (končí převodem do 1. interní formy). Syntaktická analýza dělá rozbor deklarací a příkazů (např. `a = b + ;` zda je korektní). Sémantická analýza řeší smysl (např. zda v `a = b + c` jsou korektní datové typy proměnných).

3.2 Optimalizace

Optimalizace není povinná, neboť nemá vliv na výslednou funkčnost (zda dá správný výsledek). Pro zvýšení rychlosti a efektivity. Může a nemusí být závislá na cílové platformě.

Základní dělení:

- lokální
- cyklů
- globální
- mezi funkcemi
- okénková (okno má určitý počet instrukcí; *peephole*)

Příklad 2

Ukázka na výpočtu konstantního výrazu.

```
1 const float epsilon = 0.0001
2 if (u < 2 * epsilon) { }
```



Do čeho se přeloží (ne)optimalizovaný kód nahoře?

Rozebrat co v každém (i jednoduchém programu) musí překladač vykonat. Až na určení adres, výpočet výrazů, nahrazení různá atd.

3.3 Back End

Je závislá na cílové platformě (jdeme do strojového kódu). Volba příslušných instrukcí a přidělování registrů (je jich omezený počet $\Rightarrow$ efektivní využití). Okénkové optimalizace se provádí v této části. Prochází se krátké sekvence instrukcí následujících ihned za sebou, kde se dělají transformace.

3.4 Křížový překladač

Překladač, který generuje cílový program pro jinou platformu, než tu na které běží. Přeložený program se už přenáší do cílového systému hotový.

3.5 Tabulka symbolů

Jsou do ní ukládány informace z deklarativních částí programu. Např. jména, datové typy, počet dimenzí pole, deklarace funkcí jejich hlavičky, ...

3.6 Interpret

Na rozdíl od překladače negeneruje cílový kód, ale zdrojový program přeloží do interní formy a následně interní formu interpretuje (udělá výpočet). Následuje po části *Front End*. Oproti překladači je výrazně jednodušší. Ale oproti překladu je výrazně pomalejší. Dříve se používal i v Javě, nyní JavaScript nebo Python. Programy v interpretovaných mají lepší přenositelnost.

3.7 Chyby při překladu

Chyby mohou být zjištěny ve všech fázích analýzy (jak lexikální, tak syntaktické i sémantické). Při nalezení chyby by měl překladač udělat následující věci:

- poskytnou informaci uživateli, kde chyby nastala
- uvést druh chyby (lepší obecnější)
- ošetřit chybu, aby mohla být dokončena fáze analýzy.

Nejobtížnější na zotavení jsou chyby nalezené při syntaktické analýze. Nalezení chyby je triviální, neboť analyzátor se zastaví. Zotavení (synchronizace vstupu s analyzátozem, tak aby se mohlo pokračovat) je výrazně těžší. Konkrétně u analýzy:

- shora-dolů (1) je neúspěšné srovnání či pro symbol na vstupu nemáme pravidlo pro expanzi
- zdola-nahoru (2) v tabulce automatu není pro symbol na vrcholu zásobníku a symbol na vstupu žádná akce (prázdné políčko).

Pro zotavení z chyby máme více možností:

1. oprava programu na syntakticky správný pomoci:
 - vynechání chybného symbolu (symbolů)
 - vložení chybějícího symbolu (symbolů)
 - kombinace obou výše zmíněných metodou
2. vynechat část programu (čím menší vynechání, tím lépe)

Nahrazování symbolů může být zrádné neboť je těžké ho udělat korektně a při nesprávné opravě může dojít k neustálému nekontrolovatelnému zvětšování.

Panický mód

Definice 3

Jde o jednoduchou metodu, která stanoví *význačné symboly* v jazyce, které fungují jako omezovače. Při nalezení chyby se poté vynechá kód až do nalezení prvního omezovače.

3.7.1 Zotavení při analýze shora-dolů

1. Pro symboly na zásobníku najdeme *přípustné symboly*, což je sjednocení *First*-množin symbolů na zásobníku spolu se symbolem # (konec programu)
2. Vynecháme symboly dokud nenajdeme symbol, který patří přípustných symbolů
3. Synchronizujeme automat analyzátoru pomocí algoritmu

```

while not synchronizován
  if na vrcholu zásobníku je terminální symbol  $t$ 
    uděláme srovnání
    if srovnání není úspěšné
      odebereme symbol  $t$  ze zásobníku (má to stejný efekt jako kdybychom ho
      vložili do zdrojového programu a provedli úspěšné srovnání)
    else // srovnání je úspěšné
      odstraníme symbol  $t$  ze zásobníku a ze vstupu
      synchronizován  $\leftarrow$  true
  else // na vrcholu zásobníku je neterminální symbol  $A$ 
    if není pravidlo expanze pro  $A$  při daném symbolu na vstupu
      odebereme neterminální symbol a nahradíme ho na zásobníku nejkratším
      řetězcem terminálních symbolů, které lze v gramatice z něho odvodit
    else // je pravidlo expanze pro  $A$ 
      uděláme expanzi

```

Obrázek 1: Algoritmus pro synchronizaci analyzátoru

Příklad 4

V prezentacích

3.7.2 Zotavení při analýze zdola-nahoru

1. Stanovíme význačné neterminály (z pravidel pro deklarace)
2. Pro každý z těchto symbolů přidáme pravidlo $A \rightarrow$ chyba
3. Při chybě odstraníme ze zásobníku symboly dokud nenajdeme zásobníkový symbol s položkou tvaru $[B \rightarrow \alpha A. \beta]$, kde A je nějaký z význačných neterminálů
4. Pak se podle chybového pravidla přidá na zásobník symbol s položkou $[B \rightarrow \alpha A. \beta]$. Ten najdeme v tabulce automatu na řádce zásobníkového symbolu a sloupci pro symbol A .

Příklad 5

V prezentacích

4 Regulární gramatiky

Označovány jako typ 3. Jedná se o nejjednodušší formu (kladeno nejvíce nároků na pravidla).

Pravidla: pro gramatiku $G = (N, T, P, S)$

$A \rightarrow aB$

$A \rightarrow a$

Věta

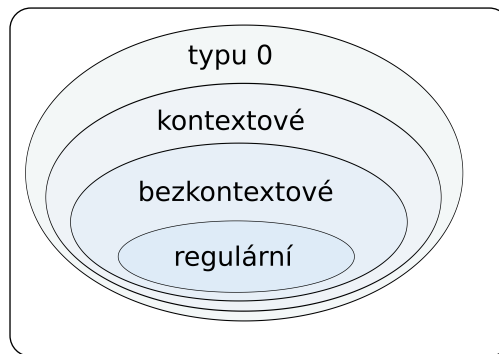
Theorem 6

Ke každé regulární gramatice G existuje deterministický konečný automat A , který rozpozná jazyk generovaný gramatikou G . Platí tedy $L(A) = L(G)$.

Příklad regulární gramatiky

Příklad 7

seznam pravidel, terminálů, neterminálů, atd.



Obrázek 2: Chomského hierarchie.

4.1 Regulární výrazy

Pro sestavení regulárních výrazů nad T , což je abeceda terminálů platí pravidla:

- každý symbol $x \in T$ je regulární výraz
- prázdný řetězec ε je regulární výraz
- jsou-li α a β regulární výrazy, pak jsou regulární výrazy i $\alpha\beta$, $\alpha \mid \beta$, α^+ a (α)
- pro a^* platí, že je to $\varepsilon \mid a^+$

[Opět možno uvést příklad terminálů a z něj regulárních výrazů, případně automat]

Konečný automat pro čísla v jazyce C

Příklad 8

[TBA]

5 Lexikální analýza / analyzátor

Jedná se o první část překladače. Průběh je takový, že čte zdrojový program a vykonává činnost, konkrétně:

- rozpozná atomy (lexikální symboly) zdrojového programu
- program převede do 1. interní formy (posloupnost symbolů)
- vynechá všechny pro překlad nepotřebné části (formátování kódu, poznámky a komentáře).

Prostě to pak vypadá jak kdybychom klasický kód hodily na jediný řádek.

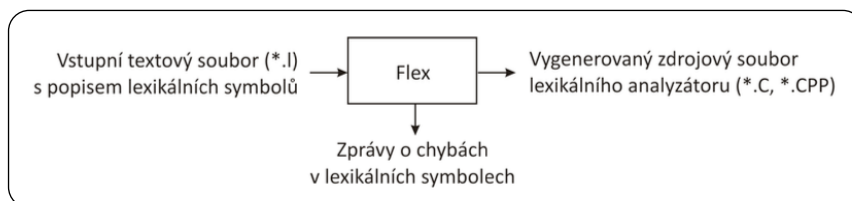
Atomy (lexikální symboly) jsou:

- klíčová slova – `int`, `typedef`, `enum`, `class`, `if`, `for`, `return`
- identifikátory – `i`, `suma`, `pocet`
- literály – `12` `1.E5` `"Evzen"`
- oddělovače – `,`
- omezovače – `[]` `{}` `()` `;`
- operátory – `+` `-` `*` `++`
- další věci

Analýza syntaxe je rozdělena na dvě části (lexikální a syntaktickou analýzu) kvůli zjednodušení. Lexikální analyzátor funguje jako konečný automat. Syntaktický analyzátor už pracuje s 1. interní formou a tím je jednodušší generovaná bezkontextová gramatika.

V překladači se lexikální analyzátor nachází jako jedna funkce. Ta se volá vždy když navazující syntaktický analyzátor potřebuje další symbol pro analýzu.

Lexikální analyzátor je možné realizovat dvěma způsoby (1) ručně ho sestavit nebo (2) použít generátor lexikálních analyzátorů (program *Lex* či *Flex*).



Obrázek 3: Fungování analyzátoru Flex.

5.1 Interní forma posloupnost symbolů

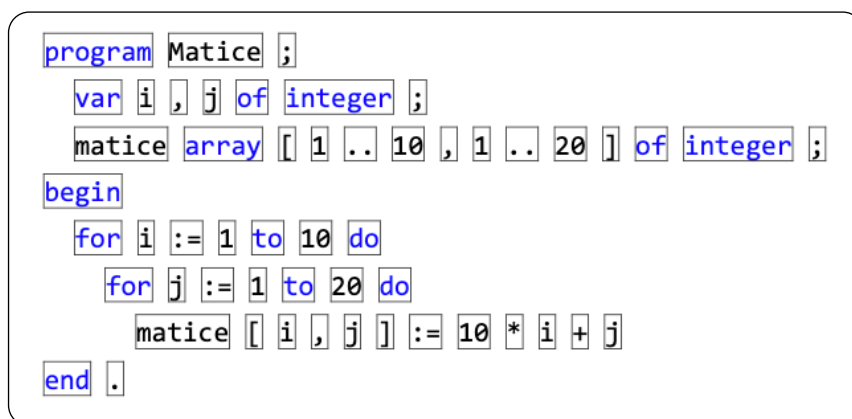
Ke každému druhu lexikálního symbolu je v daném jazyce přiřazeno celé číslo. Obvykle se pro jednoznačné symboly používá číslo z ASCII, pro víceznačné se používají čísla nad ASCII (vyšší než 255). Víceznačné jsou třeba operátory `==` nebo `<=<`.

Pokud má symbol kromě typu i hodnotu (čili identifikátor nebo literál) lexikální analyzátor předává navíc i jeho hodnotu. U literálů převede hodnoty do příslušné interní formy (např. `12` je `int`, což bude běžně 32 bitové číslo nebo znak `"A"` je `char` a ten se převede do ASCII hodnoty a podle typu kódování systému se hodnota uloží na správný počet bitů).

5.2 Rozpoznání konce lexikálního symbolu

Ve zdrojovém programu jsou jednotlivé symboly za sebou $\Rightarrow$ lexikální analyzátor je musí korektně oddělit (zjistit kde končí). Používá se způsob kdy lexikální analyzátor k právě rozpoznanému symbolu připojuje další následující znaky dokud vzniká korektní lexikální symbol (jakéhokoliv typu).

Např. výraz `b+++c` lze interpretovat dvěma způsoby (a) `b++ + c` nebo (b) `b + ++c`. Lexikální analyzátor zvolí variantu (a) neboť operátor `++` za `b` je delší než `+`.



Obrázek 4: Rozdělení kódu v Pascalu lexikálním analyzátozem.

5.3 Klíčová slova v programu

Obvykle se pro zápis klíčových slov používá stejná syntaxe jako pro identifikátory. Lze tedy zvolit dva přístupy k jejich rozpoznání:

1. klíčová slova rozpozná lexikální analyzátor
2. lexikální analyzátor je předa jako běžné identifikátory a jsou rozpoznána až v syntaktickém analyzátozu

U druhého případu (syntaktický analyzátor) se používá hashovací tabulka (minimální s perfektním hashováním), kde po rozpoznání identifikátoru je možná rychle zjistit zda se jedná o klíčové slovo.

V jazyce C se používá hashovací funkce $h(z_1, z_2, \dots, z_k) = \text{kod}(z_1) + \text{kod}(z_k) + k$, kde *kod* označuje, že jednotlivá písmena nejsou kódována obvyklým způsobem, ale podle speciální tabulky.

5.4 Chyby rozpoznávané při lexikální analýze

Rozpoznány jsou chyby jako:

- identifikátor má větší počet znaků než jazyk připouští
- chybný zápis čísla
- číslo má nepřipustnou velikost hodnoty
- chybný zápis znaku
- chybný zápis řetězec
- řetězec má nepřipustnou délku
- na začátku dalšího symbolu je znak, který netvoří žádný lexikální symbol (v C je to `&` a `@`)

6 Syntaktická analýza

Syntaktický analyzátor rozpozná syntaktické celky programu (deklarace, výrazy, příkazy) a ty předá dál k sémantické analýze.

Syntaxi programovacího jazyka nelze popsat regulární gramatikou. Proto se pro tento popis používají bezkontextové gramatiky (typ 2). Syntaktický analyzátor je založen na deterministickém zásobníkovém automatu. Při této analýze se sestavuje derivační strom, který odpovídá překládanému programu. Máme 2 základní způsoby sestavení stromu – směrem dolů od kořene stromu a směrem nahoru od listů. Tyto konstrukce můžeme provést pouze pro určité podtřídy bezkontextových gramatik.

Poznámka 9

U analýzy shora-dolů $LL(k)$ je kořenem stromu počáteční symbol gramatiky. U analýzy zdola-nahoru $LR(k)$ jsou listy symboly interní formy, které vytvořil lexikální analyzátor.

LL jako left to right a left most.

LR jako left to right a right most.

6.1 Syntaktická analýza shora-dolů (LL)

Používá se zásobníkový automat, který nevyužívá stavy, ale pouze zásobník. Pracuje s vrcholem (v každém kroku odebrán právě 1 symbol). Pro zásobníkové symboly se používá množina $N \cup T$ z výchozí gramatiky. Analýza se skládá ze 2 operací: expanze a srovnání.

6.1.1 Expanze

Operace probíhá v případě, kdy na vrcholu zásobníku je neterminální symbol. Expanze probíhá nahrazením tohoto symbolu pravou stranou příslušného pravidla.

6.1.2 Srovnání

Provádí se v případě kdy je na vrcholu zásobníku terminální symbol. Symbol je srovnán se symbolem na vstupu a pokud jsou shodné, tak se oba odstraní.

Automat rozpozná správný zdrojový program, pokud ho přijme celý, což se prokáže prázdným zásobníkem na konci.

Z třídy gramatik $LL(k)$, které umožňují deterministickou analýzu se používají v překladačích výlučně $LL(1)$. Což znamená, že se čte zleva doprava, používá se levá derivace a stačí znát jen jeden symbol vstupního řetězce (ten který je právě na vstupu).

LL(1) gramatika

Definice 10

Bezkontextová gramatika je $LL(1)$ gramatikou, jestliže každá 2 její pravidla se stejným symbolem na levé straně

$$A \rightarrow \alpha$$

$$A \rightarrow \beta$$

splňující podmínku disjunktnosti množin *First-Follow*

$$\mathbf{First(a\ Follow(A))} \cap \mathbf{First(b\ Follow(A))} = \emptyset$$

Množina First

Definice 11

Definována předpisem

$$\mathbf{First}(\alpha) = \{t \mid \alpha \xRightarrow{*} t\mu, t \in T\} \cup \{\varepsilon \mid \alpha \xRightarrow{*} \varepsilon\}$$

Jedná se o množinu všech terminálů (α), které se mohou objevit na začátku řetězce použitím libovolných pravidel.

Množina Follow

Definice 12

Definována předpisem

$$\mathbf{Follow}(A) = \{t \mid S \xRightarrow{*} \mu Av, v \neq \varepsilon, t \in \mathbf{First}(v)\} \cup \{\varepsilon \mid S \xRightarrow{*} \mu A\}$$

Množina všech terminálů, které se mohou kdykoliv vyskytnout bezprostředně za neterminálem A .

6.1.3 Konstrukce zásobníkového automatu pro gramatiku $LL(1)$

Postup je popsán v krocích:

1. pravidla gramatiky očíslovíme
2. pro každé pravidlo $A \rightarrow \alpha$ vypočítáme množinu $\mathbf{Firs(a\ Follow(A))}$
3. sestavíme tabulku automatu (řádky jsou neterminály gramatiky, sloupce jsou terminály a ε)
4. jednotlivé pozice vyplníme tak, že do pozice, která je na řádku s neterminálem A a v sloupci se symbolem t napíšeme číslo pravidla (případně celé pravidlo), právě když t patří do množiny **First-Follow** tohoto pravidla (pokud neexistuje, tak pozice zůstane prázdná)

6.1.4 Činnost automatu

1. na počátku je vstupem řetězec terminálních symbolů gramatiky (interní forma zdrojáku, kterou vytvořil lexikální analyzátor) a na vrcholu zásobníku je počáteční symbol gramatiky
2. v každém kroku se provede *expanze* nebo *srovnání* podle toho co je na vrcholu zásobníku (viz 6.1.1 a 6.1.2)

3. když zásobník zůstane prázdný je zdrojový kód syntakticky správný

Příklad 13

[V PDF prekladace03 na straně 6.]

6.2 Konstrukce analyzátoru metodou rekurzivního sestupu

Pro každý neterminál A sestavíme jednu funkci, která udělá analýzu podle všech pravidel gramatiky s tímto symbolem na levé straně.

Značení

u ... proměnná, ve které je lexikální symbol naposledy předaný lexikálním analyzátozem

`lex()` ... funkce lexikálního analyzátoru

`chyba()` ... funkce pro ošetření (syntaktické) chyby v předkládaném programu

Poznámka 14

Pro pravidla se symbolem A na levé straně se spočítají jejich množiny *First-Follow* jako:

$A \rightarrow \alpha_1 \dots \text{First}(\alpha_1 \text{Follow}(A)) = \{a_{11}, a_{12}, \dots, a_{1p}\}$

$A \rightarrow \alpha_2 \dots \text{First}(\alpha_2 \text{Follow}(A)) = \{a_{21}, a_{22}, \dots, a_{2p}\}$

až po k .

Funkce pro analýzu pak vypadá takto:

```
1  ParseA()  
2  { switch (u) {  
3      case a11:  
4      case a12:  
5          ...  
6      case a1p: // akce pro  $\alpha_1$   
7      case a21:  
8      case a22:  
9          ...  
10     case a2q: // akce pro  $\alpha_2$   
11     ...  
12     case ak1:  
13     case ak2:  
14     ...  
15     case akr: // akce pro  $\alpha_k$   
16     default: chyba()  
17     }  
18 }
```

Poznámka 15

Symbol # označuje konec analyzovaného programu (nahrazuje ε).

Pro gramatiku s pravidly:

$$E \rightarrow TF$$

$$F \rightarrow +TF \mid \varepsilon$$

$$T \rightarrow i \mid (E)$$

máme takovýto program:

```

1  Srovnani(v)
2  {
3  if (u==v) u=lex(); else chyba();
4  }
5  ParseE()
6  {
7  switch (u)
8  { case '(':
9    case 'i': ParseT(); ParseF(); return; // E→TF
10   default: chyba(); }
11 }
12 ParseF()
13 {
14 switch (u) {
15   case '+': u=lex(); ParseT(); ParseF(); return; // F→+TF
16   case ')':
17   case '#': return; // F→ε
18   default: chyba(); }
19 }
20 ParseT()
21 {
22 switch (u) {
23   case 'i': u=lex(); return; // T→i
24   case '(': u=lex(); ParseE(); Srovnani(')'); // T→(E)
25   return;
26   default: chyba(); }
27 }
```

6.2.1 Problémy

Může nastat, že některá pravidla popisující daný programovací jazyk nevyhovují pravidlům LL(1) gramatiky. Jsou možná řešení:

1. konflikty odstranit transformacemi nebo
2. konflikty v gramatických pravidlech ošetřit ručně při sestavování funkcí pro rekurzivní sestup.

Postup transformace

Mělo by být v transformace LL(1).pdf

6.3 Syntaktická analýza zdola-nahoru

Opět se používá zásobníkový automat, který využívá pouze zásobník. Specifická varianta která pracuje s celým zásobníkem = při přechodu může být odebrán **libovolný počet** symbolů. Konstrukce automatů pro tuto analýzu je značně složitější než pro analýzu shora-dolů.

Na začátku je zásobník prázdný. Pokud je vstupní věta rozpoznána, obsahuje počáteční symbol gramatiky.

Analýza se skládá ze 2 operací:

- **přesun** ... symbol se přesune ze vstupu na zásobník
- **redukce** ... na zásobníku nahradíme pravou stranu pravidla za neterminální symbol, který je na straně levé

Ukázka operací a stavu vstupu a zásobníku

Příklad 16

vstup	zásobník	operace
$(i + i) * i - i$		přesun
$i + i) * i - i$	(	přesun i
$+i) * i - i$	(i	redukce podle $E \rightarrow i$
$+i) * i - i$	(E	přesun $+$
$\vdots$	$\vdots$	$\vdots$

Deterministickou analýzu zdola-nahoru umožňuje třída LR (L ... vstupní řetězec se čte zleva doprava a R ... třída je založena na pravé derivaci). Tato třída má 4 podtřídy – $LR(0)$, $SLR(1)$ ², $LALR(1)$ ³ a $LR(1)$. Čím více je třída na začátku tím je menší a má jednodušší automaty, čím více je na konci tím je větší a má složitější automaty. Číslo v závorce určuje kolik symbolů ze vstupu je nutné znát pro deterministickou činnost.

6.3.1 Konstrukce automatu pro $SLR(1)$ gramatiku

Pro takovýto deterministický automat musíme zásobníkové symboly sestavit jako množiny podle pravidel gramatiky (nestačí pouze její terminály a neterminály). Výchozí gramatiku G si rozšíříme o $N' = N \cup \{S'\}$ a $P' = P \cup \{S \rightarrow S'\}$. Tím si usnadníme zjištění kdy je činnost ukončena (z pohledu stromu se přidá 1 uzel nad kořen – bude to nový kořen).

Zásobníkové symboly nazveme jako **množiny položek pravidel gramatiky**. Takováto položka je zápis pravidla s vyznačením místa, kde je právě fáze přesunu. Tvar vypadá takto $[A \rightarrow \mu.v]$. Tečka na pravé straně pravidla (tedy vpravo od šipky) odděluje již přesunuté symboly (tady μ) od nepřesunutých (tady v). Pro pravidlo $A \rightarrow aBC$ máme tedy 4 možné kombinace $[A \rightarrow .aBc]$ (začátek), $[A \rightarrow a.Bc]$, $[A \rightarrow aB.c]$, $[A \rightarrow aBc.]$. Specifická jsou pravidla typu $A \rightarrow \epsilon$, ty jsou ihned redukční.

Při sestavování nového zásobníkového symbolu se vytváří **uzávěr** $U(M)$ výchozí množiny položek gramatiky M . Do uzávěru se pro každou položku v M , která má právě na přesunu neterminální symbol, přidají výchozí položky všech pravidel s tímto neterminálním symbolem na levé straně. Jinak řečeno jde o pravidla jejichž redukcí lze realizovat přesun daného neterminálního symbolu. Výpočet může mít tedy rekurzivní charakter, neboť nově přidaná položka může mít na přesunu (tedy za tečkou) opět neterminální symbol.

Uzávěr $U(M)$

Definice 17

$$U(M) = M \cup \{[B \rightarrow .\beta] \mid [A \rightarrow \mu.Bv] \in U(M), B \rightarrow \beta \in P\}$$

[V krocích lze velmi přesně popsat postup sestavení automatu].

[Taktéž lze v krocích popsat činnost automatu.]

²Simple

³Look-Ahead

Větná forma ... řetězec, který lze odvodit z počátečního symbolu gramatiky (průběžný krok věty)

Věta ... řetězec terminálů, který lze odvodit z počátečního symbolu

Fáze ... levá část větné formy

Prediktivní množina

Definice 18

Prediktivní množina $\omega \subseteq T \cup \{\#\}$, kde $\#$ je konec překládaného programu. Jde o množinu, která se používá pro rozhodnutí kdy lze provést operaci redukce.

6.3.2 Konstrukce automatu pro LALR(1) gramatiku

Výchozí gramatiku G si opět rozšíříme o $N' = N \cup \{S'\}$ a $P' = P \cup \{S \rightarrow S'\}$ (tedy stejným způsobem jako pro $SLR(1)$ gramatiku).

Zásobníkové symboly nazveme opět jako **množiny položek pravidel gramatiky**. Jejich tvar je ale lehce odlišný, konkrétně navíc obsahuje prediktivní množinu symbolů (ω) používanou pro rozhodnutí kdy lze provést operaci redukce. Zápis má tvar

$$[A \rightarrow \mu.v; \omega]$$

kde $\omega \subseteq T \cup \{\#\}$ ($\#$... značí konec překládaného programu).

Pravidlo se může opět nacházet ve 4 různých fázích.

Pokud je řetězec $v = a\eta$, kde $a \in T$ (terminál) je situace zřejmá. Pokud však $v = B\eta$, kde $B \in N$ (neterminál) musí být proveden přesun pravé strany některého z pravidel s tímto symbolem B na levé straně a následně pak redukce pravé stran β tohoto pravidla. Položka $[B \rightarrow .\beta; \dots]$ pak bude taky obsažena v zásobníkovém symbolu.

Opět se provádí výpočet **uzávěru** $U(M)$ pro nějž platí:

$$U(M) = M \cup \{[B \rightarrow .\beta; \tau] \mid [A \rightarrow \mu.Bv; \omega] \in U(M), B \rightarrow \beta \in P, \tau = \text{First}(v\omega)\}$$

Při konstrukci automatu pro LALR(1) jsou slučovány zásobníkové symboly se stejným jádrem.

Jádro zásobníkového symbolu je tvořeno jeho položkami, ve kterých jsou vynechány prediktivní množiny. Neboli

$$z = \{[A_1 \rightarrow \mu_1.v_1; \omega_1] \dots [A_1 \rightarrow \mu_k.v_k; \omega_k]\}$$

$$\text{kernel}(z) = \{[A_1 \rightarrow \mu_1.v_1] \dots [A_1 \rightarrow \mu_k.v_k]\}$$

V tomto slučování je oproti konstrukci pro LR(1) rozdíl, neboť tam slučování neprobíhá. Automat LR(1) má tedy více zásobníkových symbolů $\Rightarrow$ zvyšuje jeho rozpoznávací schopnosti. Třídy jazyků LR(1) je tedy obsáhlejší než třída LALR(1).

Postup sestavení automatu (kroky)

1. Z pravidla s počátečním symbolem na levé straně zkonstruujeme počáteční zásobníkový symbol $z_0 = U(\{[S' \rightarrow .S; \#]\})$. Zařadíme ho do množiny Z ($Z = \{z_0\}$).
2. *lorem ipsum*
3. *lorem ipsum*
4. *lorem ipsum*
5. *lorem ipsum*
6. *lorem ipsum*

Činnost automatu (kroky)

1. Počáteční podmínky: na vstupu je vstupní řetězec a na zásobníku je symbol z_0
2. *průběžné kroky*
3. $\vdots$

Příklad činnosti automatu LALR(1)

Příklad 19

TBA

6.3.3 Nedeterminismus automatu

Výše sestavený automat je nedeterministický, neboli umožňuje více akcí, čímž vzniknou konflikty. **Konflikty** máme 2 druhů:

- **přesun-redukce** – na dané pozici automatu je akce přesunu i redukce, vznikne pokud je v zásobníkovém symbolu položka pro přesun symbolu t a také položka pro redukci symbolu t

$$z = \{ \dots [A \rightarrow \mu.tv] \dots [B \rightarrow \beta.] \dots \} \quad t \in \text{Follow}(B)$$

- **redukce-redukce** – na pozici automatu jsou 2 různé akce redukce, v zásobníkovém symbolu musí být 2 redukční položky pro stejný symbol t

$$z = \{ \dots [A \rightarrow \alpha.] \dots [B \rightarrow \beta.] \dots \} \quad t \in \text{Follow}(A) \wedge t \in \text{Follow}(B)$$

Komplikovanější konflikty

Poznámka 20

Výjimečně se mohou vyskytnout i komplikovanější konflikty jako *přesun + 2 redukce*.

6.3.3.1 Řešení konfliktů

Prakticky se používá pouze možnost **přepsání pravidel gramatiky**, které konflikt způsobují. Gramatiky obsahuje konflikty pokud **není jednoznačná**. Tento přepis obvykle udělá gramatiku rozsáhlejší.

Příklad 21

TBA

Konflikty se případně mohou řešit i ručně, kdy se pro každé políčko automatu, kde je konflikt, rozhodne co se tam má nechat a zbytek se odstraní.

Při těchto konfliktech (čili i analýzách) je vhodné si nakreslit derivační strom, který umožní na první pohled vidět, která z operací se má provést dříve a je tedy korektní. Např. víme, že obvykle je v programovacích jazycích součet levě asociativní, proto tuto variantu v tabulce ponecháme.

7 Sémantická analýza

Existují 3 způsoby popisu sémantiky: axiomatická sémantika, operační sémantika, denotační sémantika. Na rozdíl od syntaktické části je sémantická část jazyků značně odlišná a ani jeden ze 3 způsobů se neosvědčil. Pro sémantické analyzátoři se používají tedy **atributové gramatiky**.

7.1 Atributová gramatika

Jde o nadstavbu bezkontextové gramatiky (ta popisuje syntaxi). Přídavkem k této gramatice jsou sémantické informace jako atributy sémantických pravidel (např. typ nebo hodnota). Tyto atributy jsou údaje z $N \cup T$.

Symbol nemusí mít žádný atribut nebo jich může mít i více. Množinu přiřazených atributů symbolu X označíme jako $A(X)$. Zápis, že symbol má atribut vypadá $X.a$. Sémantická pravidla pro výpočet atributů jsou přiřazena pravidlům výchozí bezkontextové gramatiky.

Slouží k typové kontrole (např. zda můžeme provést součet dvou **int**).

Příklad 22

Syntaktické pravidlo:

$$X_0 \rightarrow X_1 X_2 \dots X_m$$

Jemu přiřazené sémantické pravidlo (jako funkce):

$$X_i.a = f(X_j.u, X_k.v, \dots, X_r.z)$$

Takovýto zápis říká, že atribut je počítán z jiných atributů přiřazených symbolům daného syntaktického pravidla.

Atribut má libovolný datový typ (číslo, znak, strukturovaný, ...).

Máme 2 typy atributů:

- **syntetizovaný atribut** – jde o atribut, který je počítán pro levou stranu syntaktického pravidla
- **dědičný atribut** – jde o atribut, který je počítán pro pravou stranu syntaktického pravidla

Pro každý výpočet atributu je v derivačním stromu maximálně jedno sémantické pravidlo (takže výpočet je jednoznačný).

Atributová gramatika

Definice 23

atributová gramatika: $AG = (G, A, R)$, kde

$$A = \bigcup_{X \in N \cup T} A(X) \dots \text{množina atributů}$$

R ... konečná množina sémantických pravidel

(výchozí) bezkontextová gramatika: $G = (N, T, P, S)$

$AS(X)$... množina syntetizovaných přiřazených atributů

$AI(X)$... množina dědičných přiřazených atributů

Platí, že $AS(X) \cup AI(X) = A(X)$ i $AS(X) \cap AI(X) = \emptyset$

Výpočet atributů a sestavení stromu

Příklad 24

V prezentaci

7.1.1 Dobře definovaná gramatika

Nazveme tak atributovou gramatiku, jestliže v každém derivačním stromu pro každý v něm obsažený atribut existuje 1 pravidlo pro výpočet a gramatika je acyklická.

Acyklická gramatika

Definice 25

Acyklická je gramatika pokud lze nalézt pořadí výpočtu atributů, kdy atribut a je počítán podle $X_i.a = f(X_j.u, X_k.v, \dots, X_r.z)$ ve chvíli kdy atributy $X_j.u, X_k.v, \dots, X_r.z$ jsou známy (vypočítány).

Můžeme poté říkat, že atribut a závisí na attributech, ze kterých se počítá. Acykličnost vychází z grafu přímých závislostí, který můžeme sestavit. Uzly grafu jsou atributy symbolů derivačního stromu a hrana je mezi symboly pokud $X_i.a$ přímo závisí na $X_k.b$.

Graf závislostí

Příklad 26

V prezentaci

Ověření, že je gramatika acyklická je náročné, proto se ověřuje silnější vlastnost, kterou je **silná acykličnost**. Ta platí pokud je pro každý symbol $X \in N \cup T$ stanoveno pořadí výpočtu atributů (značené jako $O(A(X))^4$) splňující grafu. Graf který sestavíme pro každé pravidlo a platí, že hrany jsou:

- z uzlu $X_j.b$ do $X_i.a$ jestliže $X_i.a$ závisí na $X_j.b$
- z uzlu $X_j.b$ do $X_i.a$ jestliže v uspořádání $O(A(X))$ je $X_j.b$ před $X_i.a$

Pak je graf acyklický.

7.1.2 L-atributová gramatika

Tato gramatika umožňuje průběžný výpočet atributů při analýze shora-dolů. Pro každé syntaktické pravidlo platí, že dědičné atributy symbolu X_j na jeho pravé straně jsou přímo závislé jen na attributech symbolů $X_1, \dots, X_{j-1}$ a dědičných attributech symbolu X_0 .

Potom můžeme atributy počítat v pořadí:

$$AI(X_0), AI(X_1), AS(X_1), AI(X_2), AS(X_2), \dots, AI(X_m), AS(X_m), AS(X_0)$$

Pro L-atributovou gramatiku $AG = (A, R, G)$ (kde G je LL(1) gramatika) můžeme sestavit deterministický sémantický analyzátor se 2 zásobníky:

- syntaktický zásobník – sem se ukládají zásobníkové symboly zásobníkového automatu
- sémantický zásobník – sem se ukládají atributy

Ukázka činnosti analyzátoru

Příklad 27

V prezentaci

7.1.3 Porovnání atributových gramatik při syntaktické analýze

Při analýze shora-dolů je ihned na začátku známo podle jakého pravidla analýza probíhá. Tím pádem jsou známa i sémantická pravidla, přiřazená k tomuto pravidlu. Takže současně při syntaktické analýze můžeme počítat i atributy jeho symbolů.

⁴ O jako order.

Zatímco při analýze zdola-nahoru při operaci přesunu neznáme syntaktické pravidlo. Takže nemůžeme počítat ani atributy. Až při stanovení pravidla redukce určíme syntaktické pravidlo a tím pak můžeme spočítat atributy.

Tedy pro výpočet atributů je lepší analýza shora-dolů.

7.1.4 S-atributová gramatika

Jde o atributovou gramatiku, která obsahuje jen syntežované atributy (ty na levé straně pravidla). Umožňuje průběžný výpočet atributů při analýze zdola-nahoru.

Analýzu zdola-nahoru zde můžeme realizovat jen s 1 zásobníkem. Tam se ukládají jak zásobníkové symboly zásobníkového automatu, tak i syntežované atributy.

Při operaci přesunu symbolu X se na zásobník uloží jeho zásobníkový symbol a množina syntežovaných atributů tohoto symbolu. Po ukončení přesunu symbolů pravé strany pravidla se vypočítají syntežované atributy symbolu X_0 (ten je na levé straně pravidla) a ze zásobníku se odstraní zásobníkové symboly odpovídající symbolům pravé strany pravidla. Na zásobník se pak uloží zásobníkový symbol pro symbol na levé straně pravidla a jeho syntežované atributy.

Příklad 28

V prezentaci

Použití pouze syntežovaných atributů nás neomezuje. Umožňují šířit informaci směrem nahoru. Pro směr dolů, který by jinak zastávaly dědičné atributy, se použijí odkazy, které říkají kam se má sémantický údaj dolů dostat.

7.2 Interní formy programu po sémantické analýze

Sémantický analyzátor generuje **druhou interní formu**. Máme 2 základní druhy interní formy – *stromovou* (AST) a *lineární* (trojice a čtveřice).

7.2.1 Stromová interní forma (AST⁵)

Je odvozena z derivačního stromu, který se vytvořil při syntaktické analýze. Vynechají se všechny části, které měly význam pouze pro syntaktickou analýzu. V podstatě se vynechají všechny uzly, kde nejsou znaky přímo z výrazu (obvykle všechny neterminály a některé terminály).

7.2.2 Lineární forma

Jedná se o běžně používanou formu, kde se používá především následující čtveřice:

operace	1. operand	2. operand	výsledek
---------	------------	------------	----------

V části operace můžeme mít jak aritmetickou, logickou či srovnání. Operandy nemusí být vyplněné. V části výsledek je místo, kam se výsledek uloží či kam se provede skok (návěští).

Pro zkrácený zápis je možné použít trojice, kde chybí část s výsledkem a místo dočasných proměnných se používá odkaz na číslo čtveřice, jejíž výsledek tvoří operand dané operace. Tato forma není vhodná pro překladače s optimalizací, neboť dochází ke změně pořadí čtveřic a to by se zde obtížně řešilo.

⁵Abstract syntax tree

7.2.3 Generování interní formy

Syntaktický analyzátor v průběhu analýzy volá **sémantické funkce**, které dělají sémantickou analýzu jednotlivých částí programu. A pro tyto části generují příslušnou interní formu. Tyto sémantické funkce používají další funkce generující dočasné proměnné, návěští a čtveřice interní formy. Konkrétně jde o:

- **newtemp()** ... generuje novou dočasnou proměnnou
- **newlabel()** ... generuje nové návěští
- **gen(-, -, -, -)** ... generuje čtveřici interní formy (**operace**, **operand1**, **operand2**, **výsledek**)

Máme i specifické atributy:

- **E.ref** ... atribut, který má uložen odkaz na operand/výsledek aritmetické/logické operace; reference může být na proměnnou, literál, dočasnou proměnnou
- **X.begin** ... návěští umístěné před výrazem/příkazem
- **X.lab** ... návěští umístěné ve výrazu/příkazu
- **X.end** ... návěští umístěné za výrazem/příkazem

<sem> ... v syntaktickém pravidle označuje místo, kde je volána sémantická funkce

Aritmetické výrazy

Příklad 29

$E \rightarrow E + E$ < sem >

- sem: **E1.ref** = **newtemp()**; **gen**('+', **E2.ref**, **E3.ref**, **E1.ref**)

$E \rightarrow E \cdot E$ < sem >

- sem: **E1.ref** = **newtemp()**; **gen**('*', **E2.ref**, **E3.ref**, **E1.ref**)

⋮

v prezentaci

Pro logické výrazy je situace o něco složitější, jsou totiž překládány dvojím způsobem.

1. jako aritmetické výrazy (např. Visual Basic nebo Pascal)
2. s využitím vyhodnocení pravdivosti spojek disjunkce/konjunkce (např. C, C#)

Druhý způsob známe jako zkrácené vyhodnocování, kde se začíná u **and** a **or** u levého operandu a pokud jeho hodnota stačí pro to abychom znali hodnotu celého výrazu vyhodnocování skončí (pravý operand se nevyhodnotí)⁶.

⁶U **or** by byl levý **TRUE** a u **and** by musel být **FALSE**.

$E \rightarrow E < \text{sem1} > \text{ and } E < \text{sem2} >$

```

• 1 // sem1
2 E1.ref = newtemp()
3 E1.end = newlabe()
4 gen(=, 0, -, E1.ref)
5 gen(cmp, E2.ref, 0, -)
6 gen(jeq, -, -, E1.end)
7
8 // sem2
9 gen(cmp, E3.ref, 0, -)
10 gen(jeq, -, -, E1.end)
11 gen(=, 1, -, E1.ref)
12 gen(lab, -, -, E1.ref)

```

*Pak následuje ještě **or**, **not**.*

Podmínkové příkazy

Příklad 31

Dělí se na 2 typy – úplné a neúplné (podle toho jestli mají **else** větev).

V prezentaci

8 Tabulky symbolů

Do těchto tabulek se ukládají údaje obsažené v deklaracích a definicích. Dále i jiné údaje odvozené při sémantické analýze a jiných částech překladu. Informace mají obecně různý charakter.

V průběhu překladu jsou tabulky velmi intenzivně využívány. Efektivnost na vyhledávání je tedy zásadní a implementovány tím pádem jsou jako hashovací tabulky.

Počet tabulek může být různý. Od jediné globální až po více vrstev lokálních tabulek, které se mohou vytvářet i při překladu každé funkce. Jména proměnných se mohou ukládat přímo do tabulky nebo na ně může být odkazováno do separátní tabulky.

Tabulky mohou mít různý rozsah platnosti, na který je nutné dbát při vyhledávání v nich. Rozsah se řídí podle pravidel daného jazyka. Tento systém je vlastně stejný jako při prostředích v PP. Prohledávání tedy vždy probíhá odspodu (nejbližší tabulka) a postupuje výše až do tabulky globální (kořenové).

U jazyků jako C to funguje trochu jinak. Tam nejsou tabulky zvlášť, ale existuje jediná tabulka pro všechny úrovně a používá se prostředek, který odstraňuje symboly deklarované v bloku na konci tohoto bloku.

9 Blok programu

Základní blok programu

Definice 32

Jde o blok programu, ve kterém není žádná instrukce větvení (takže ani skoky nebo volání podprogramu). Instrukce větvení může být pouze poslední instrukcí v bloku.

Graf toku dat

Definice 33

Jde o orientovaný graf s vlastnostmi:

- uzly jsou základní bloky programu
- hrany vyjadřují přechody (větvení) mezi jednotlivými bloky

Hrana z bloku B_i do B_k vede pokud po ukončení bloku B_i může nastat přechod do bloku B_k .

Abychom mohli takovýto graf sestavit je vhodné mít lineární reprezentaci (ony čtveřice).

Algoritmus pro sestavení grafu

1. Celý program označíme jako základní blok
2. Procházíme jednotlivé čtveřice interní formy
 - pokud je cílové návěští větvení, označíme ji jako začátek nového bloku a předchozí čtveřici jako konec předchozího bloku
 - pokud čtveřice reprezentuje instrukci větvení, tak ji označíme jako konec stávajícího bloku a následující čtveřice bude začátkem dalšího bloku

Ze základních bloků sestavíme uzly, následně tyto bloky projdeme a doplníme hrany. Vpodstatě je to velmi jednoduché, při podmínkách **if else** je určení bloků a hran intuitivní.

Příklad 34

```
1 if (i != j) {  
2     if (j > 0) i = j;  
3     j = j + 1  
4 }  
5 else {  
6     j = 2 * i  
7 }  
8 k = i + j
```

C++

10 Optimalizace programů

Máme 2 základní přístupy k optimalizaci – **lokální** a **globální**. Lokální je jednodušší a optimalizuje jednotlivé základní bloky programu. V globální se navíc optimalizují i toky dat mezi jednotlivými bloky. Globální optimalizace je náročnější, ale u účinnější.

10.1 Co vše můžeme optimalizovat

Některé optimalizace jde provádět na obou úrovních a jedná se o základní typy, které jsou dále vyjmenované.

Konstantní výrazy (podvýrazy) – jsou to ty, které obsahují operace, jejichž hodnoty operandů jsou konstantní (lze je stanovit přímo ze zdrojového kódu); jejich výpočet v době překladač je výhodnější

Poznámka 35

Dva výrazy (podvýrazy) považujeme za stejné pokud mají stejnou operaci a operandy mají prokazatelně stejnou hodnotu.

Superlokální optimalizace – zvyšuje účinnost lokální optimalizace tím, že optimalizuje rozšířené bloky

Rozšířený blok

Definice 36

Rozšířený blok je posloupnost základních bloků, kde platí, že kromě prvního bloku má každý blok jen 1 předchůdce.

11 Generování kódu

Tato činnost překladače souvisí s pipeline architekturou procesorů. V ní je vykonávání strojové instrukce rozděleno na pevný počet fází. Tento počet je obvykle 5. Jedná se o tyto:

1. Načtení instrukce
2. Dekódování operačního kódu
3. Výpočet aritmetické nebo logické operace (pro instrukce součtu, rozdílu atd.) nebo výpočet adresy pro instrukce čtení/zápisu z/do paměti
4. Přístup do paměti
5. Zápis výsledku aritmetické nebo logické operace do registrů

Těchto 5 různých instrukcí se může provádět najednou. Některé instrukce nemusí využívat všech 5 fází.

Tato pipeline architektura přinesla problém, kdy instrukce mohou být na sobě **datově závislé**. Jako příklad můžeme uvést následující kód, kde je operace `*` závislá na operaci `+`.

```
1 d = (a+b) * c
```



11.1 Přidělování registrů

Jelikož registr je nejrychlejší místo, kam se může přistupovat pro data, je nutné stanovit jejich efektivní využití (ovlivňuje program). Počet registrů je omezený. Optimální využití je problém NP-úplný a pro jeho řešení se tedy používají heuristické algoritmy. Dnes je nepoužívanější metoda založená na barvení grafu.

11.1.1 Přidělování metodou barvení grafu

Výchozí podmínky jsou:

- mít graf inferencí I
- k dispozici je m registrů (barev) a máme posloupnost $1, 2, \dots, m$

Algoritmus probíhá následovně:

1. G je úplný podgraf grafu I , z G odebíráme uzly (včetně hran) a ukládáme na zásobník Z následujícím způsobem
2. je-li v G uzel se stupněm $< m$ odebereme ten a uložíme ho na zásobník

3. není-li v G takový uzel, odebereme libovolný a jdeme zpět na 2.
4. po uložení všech uzlů na zásobník začneme uzly z něj odebírat a barvíme je způsobem viz dále
5. odeberme uzel u ze zásobníku, najdeme barvu s nejmenším číslem takovou, že s ohledem na sousedy je možné uzel obarvit
6. pokud vhodná barva neexistuje, vezmeme z dosud neobarvených uzlů uzel s největším stupněm a odstraníme ho z grafu I (taková hodnota bude uložena v paměti)
7. celý proces opakujeme od 1. kroku

Po skončení algoritmu jsou obarvené uzly, ty které mohou být uloženy v registrech, ostatní uzly musí mít hodnotu uloženou v paměti.